ATR Rebuild Report

Moving the agentic-tool-reasoning pipeline from a 6-tool logistics world to the single-tool BM25 MuSiQue retrieval contract.

Date: 29 Aug 2026 | Scope: schema, world, tools, generator, verifiers, tests, and the 12k training dataset.

1. The big picture: why this rebuild happened

The benchmark organiser (the judge) finalised the task definition, and it changed what our system has to do. The most important consequence is the collapse of the tool set.

	Before (old world)	After (new contract)
Tools	6 tools: web_search, fetch_page, db_query, db_aggregate, calculator, search_docs	1 tool: search(q) → BM25 ranked retriever over a provided candidate
Domain	Logistics / products / warehouses	Wikipedia-style encyclopedic entities (people, places, works, orgs
Question type	Single-fact lookups + some multi-hop	MuSiQue-style multi-hop retrieval (2/3/4-hop chains)
Answer source	Internal database + web	Passages supplied with each question (retrieved hop by hop)
Headline metric	Exact / normalised match	Token-F1 (matches the ~34% baseline; target ~60%)

What this means in plain English

The whole task is now: **read a multi-hop question, and use one search tool (BM25) over the passages given to you to find the answer, one hop at a time.** The calculator, database, web search and page-fetch tools are gone. This is exactly the shape of the public MuSiQue benchmark the judge uses.

2. What changed in each file

2.1 atr/tools/world.py — the encyclopedic world

Replaced the logistics domain (products, warehouses, shipments) with a seeded, deterministic **encyclopedic** entity universe. Each entity gets a Wikipedia-style passage (title + text + facts + links). The 'links' are what create multi-hop questions: a person passage links to a company, the company links to a city, the city to a country.

Entity kinds in the world:

- **Countries** — name, capital, region, population, official language (6 countries)
- **Geographic features** — mountains, rivers, bays, mountains (5 per seed)
- **Cities** — the capital city of each country, with population and founding year
- **Organisations** — company name, industry field, HQ city, founding year (8 per seed)
- **People** — name, profession, birth city, employer (10 per seed)
- **Works** — novel / film / symphony, publishing year, author (3 per seed)

Every episode builds its **own world from a seed**, so tools are executable and stateful without leaking between tasks. The task generator reads the same world to compute the gold answer, which is what makes the verifiers *executable* rather than string-matched against a frozen answer key.

Key design idea — the swap-out point

This file (and builtin.py) are explicitly marked as the **only two files that get replaced** when the organiser's real tool environment arrives. Everything downstream talks to the ToolRegistry, not to the World directly.

2.2 atr/tools/builtin.py — the single BM25 search tool

Replaced the 6-tool set with exactly one tool, **search**. It is a genuine BM25 ranker (real TF x IDF x length-normalisation) over the candidate passages for the episode. Title matches get a small 1.6x boost because title words are strong signals for identifying the right entity.

The search tool's behaviour:

- **Empty query** → a readable ToolError ('query is empty') telling the model to add terms
- **No matches** → returns an empty result set (legitimate; the model must rephrase)
- **Matches** → returns full passage texts (title + text), so a retrieved passage can directly supply both the next hop's query and, at the leaf, the final answer

What was removed

- calculator — arithmetic is now a no_tool-style skill, not a tool
- web_search / fetch_page — there is no external web; only the given passages
- db_query / db_aggregate — the database is gone; the 'tables' are now just the documents

2.3 atr/tasks/schema.py — the task families

The TASK_TYPES list now contains only the MuSiQue-shape and negative families:

Family	What it tests	Difficulty / tier
musique_2hop	A chain of 2 links; one search resolves it	2
musique_3hop	A chain of 3 links; two dependent searches	3
musique_4hop	A chain of 4 links; three dependent searches	4
unanswerable	The info genuinely is not in the passages; must abstain	2
no_tool	Answerable from the model's own knowledge; a tool call is wrong	0

The old families (compute, db/doc multi-hop, recovery, action) were removed. The recovery family was dropped because it could not be made unambiguously verifiable (explained in section 3).

2.4 atr/tasks/generator.py — the MuSiQue question generator

This is the heart of the rebuild. It now mints multi-hop retrieval questions programmatically from the encyclopedic world, with no hand-labelling. Every task carries:

- **a prompt** — a nested MuSiQue-style question, e.g.
- "What is the official language of the country that contains the city where the company that employs the person who created Glass Rivers is headquartered?"
- **a gold answer** — computed from the same world the tool reads
- **an oracle_plan** — the reference sequence of search calls (one per hop)

How a chain is built

A 'route' is a list of relation steps. Each step follows a fact that names the next entity in the chain. For example the route *person* → *org* → *city* → *country* builds a question whose answer needs 4 passages read in sequence. A 4-hop question therefore needs **three dependent searches** — you cannot even write the second query until the first passage has been read. This dependency is exactly what forces (and teaches) sequential retrieval.

Example routes (2-, 3-, 4-hop)

Hops	Relation steps (chain)	Leaf / answer
2-hop	feature → country → city	a city's population or founding year
2-hop	person → org → city	a city attribute
3-hop	feature → country → city → country	a country attribute
3-hop	work → person → org → city	a city attribute
4-hop	feature → country → city → country → city	a city attribute
4-hop	work → person → org → city → country	a country attribute

Data mix

The default mix weights the set toward multi-hop work (where score is won) while keeping the negative families as cheap insurance against the two default failure modes: calling a tool for everything, and inventing an answer rather than abstaining.

Family	Share

musique_2hop	22%
musique_3hop	24%
musique_4hop	18%
unanswerable	11%
no_tool	25%

no_tool bank

The no_tool family was rebuilt from scratch (~400 generic items) using domain-independent general knowledge: capitals of real countries, chemical symbols, maths, spelling, sorting. Crucially, none of this information is present in the world, so a tool call cannot help.

3. What was removed, and why

The recovery family

The old recovery family generated tasks where a deliberately garbled first query should return nothing, forcing the model to rephrase. We removed it because it could not be made unambiguously verifiable: distinctive entity names almost always match the BM25 index, so an empty first query was essentially impossible to arrange reliably. Trying to force one with near-miss or ambiguous entities added complexity faster than it added signal. Self-correction from empty retrieval remains a supportable skill (the empty-result path is still modelled in the tool), but there is no dedicated training family for it right now.

The calculator and database tools

Removed along with the whole logistics world. Under the new contract there is no arithmetic tool, no database, and no external web — only BM25 retrieval over the provided passages. Arithmetic needed by no_tool-style questions is answered from the model's own knowledge instead of a tool.

4. A real bug found and fixed during the rebuild

While re-running the test suite we caught a genuine determinism bug. In gen_musique the code originally did:

```
routes = _ROUTES[hops]
rng.shuffle(routes)   # BUG: shuffles the GLOBAL _ROUTES in place
```

Because _ROUTES is a module-level dictionary of lists, shuffling the returned list mutated the shared global in place. That meant two calls to generate(12000) with identical seeds could pick different routes and produce different data — silently breaking reproducibility of the training set. The fix copies the list before shuffling:

```
routes = list(_ROUTES[hops])   # copy: shuffle mutates in place
rng.shuffle(routes)
```

We verified the fix: two identical generate(40) calls now produce exactly 0 differing records.

5. Tests updated (and now green)

Two test files still asserted the old 6-tool contract and would have failed. They were updated to the new single-tool / MuSiQue shape:

File	What was changed
tests/test_pipeline.py	Toolset assertion now checks the active toolset is exactly ['search']; verifies only MuSiQue + negative fam
tests/test_fix2.py	db_lookup phrasing-variation check replaced with musique_3hop variation across seeds; the old CITIES p

Measured results (the 'accuracy' of the rebuild)

Check	Result
Oracle eval on dev set — success / final / selection / args / necessity	1.0 / 1.0 / 1.0 / 1.0 / 1.0
Full 12k dataset — oracle success	12000 / 12000
tests/test_pipeline.py	ALL PASS (26 checks)
tests/test_fix2.py	ALL PASS (23 checks)
Determinism (two identical generate(40) runs)	0 differing records
Downstream: raw → reject → SFT export	12000 → 2000 kept, balanced 400 / family, 0 skippe
Tool calls in the exported SFT set	100% search (4000 calls)
Old-world residue in the new dataset	0 (no web_search / db_query / calculator / Titan Loo

*Honest caveat: these numbers measure the **oracle / verifier correctness and pipeline integrity** on headless CPU. They are not a model score — no real model has been trained or evaluated yet.*

6. Dataset regenerated

The SFT training dataset (artifacts/raw_oracle.jsonl) was regenerated from the new world/generator to match the single-BM25 search contract. The full pipeline was validated end to end:

- **Generate** — 12,000 tasks from seeds 0..11999 using the new mix
- **Oracle replay** — every task run through the oracle backend; 12,000 / 12,000 succeeded
- **Rejection filter** — 12,000 → 2,000 kept, re-balanced to 400 per family (quality checks dropped 2,202 repeated-call shapes; shape-cap dropped 7,798)
- **SFT export** — 2,000 canonical records written, 0 skipped; all tool calls are search

Artifacts on disk

File	Contents
artifacts/raw_oracle.jsonl	The new 12k single-BM25 dataset (regenerated)
artifacts/raw_oracle.6tool.bak.jsonl	Backup of the old 6-tool dataset (kept for reference)
artifacts/raw_oracle.5tool.bak.jsonl	Earlier backup (5-tool interim world)

7. What's still open (next steps)

- **Token-F1 calibration** — the judge scores by token-F1; our verifier still uses exact / normalised match. Calibrating the two on the benchmark's own 54 examples is the highest-value next step before deciding if we add an F1 verifier.
- **Harness wire format** — how the organiser exposes the candidate set to the model (the exact tool-call syntax). This is the last unknown for adapter.py; the world/generator already match the BM25-over-candidate-set shape.
- **3 / 4-hop phrasing variety** — occasional template-y wording in the longest chains (e.g. 'the capital of the country that contains the capital...') could be varied for better generalisation.

— End of report —